COMPUTER ENGINEERING

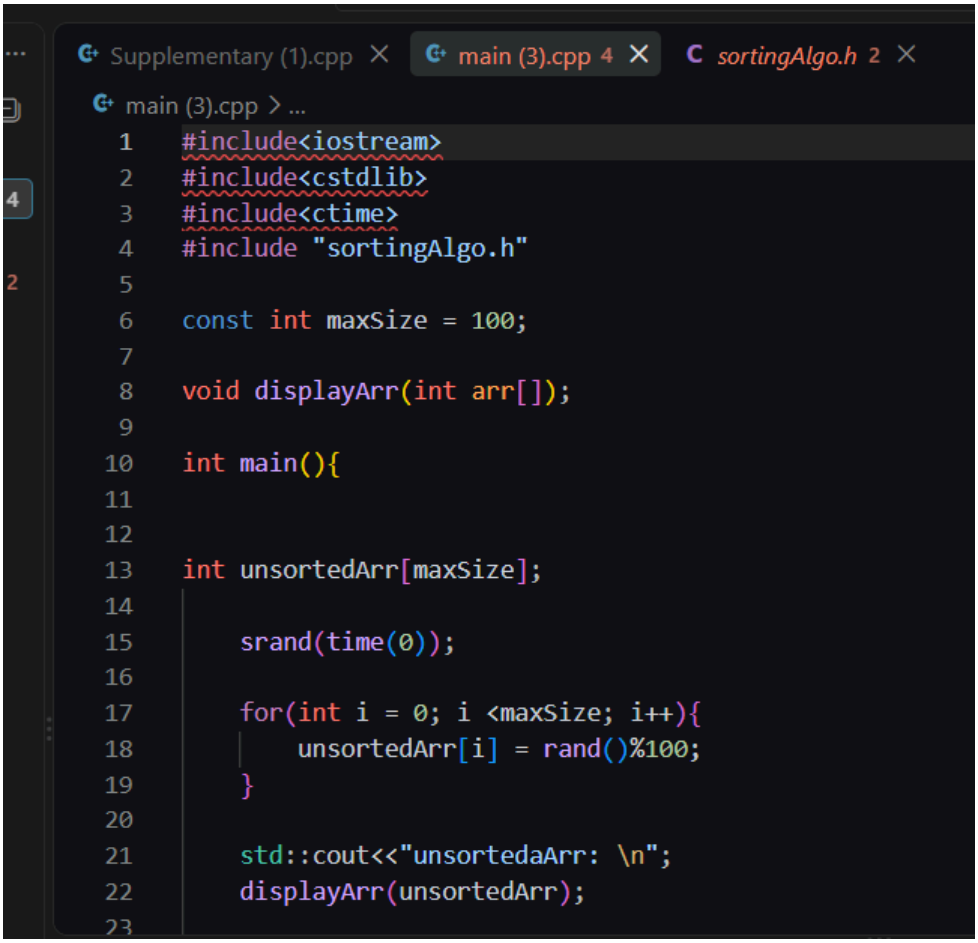
QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE010 - Data Visualization and Analysis | **SECTION:** | CPE21S2 |
| **DATE PERFORMED:** | 9/4/2026 | **DATE SUBMITTED:** | 9/8/2026 |
| **NAME:** | Estorco, Clint Hoven A. | **INSTRUCTOR:** | Engr. Jimlord M. Quejado |

**ACTIVITY NO. 6** **HOA 6 - Searching Techniques**

## 1. PROCEDURE OUTPUT

Screenshot:



```cpp
#include<iostream>
#include<cstdlib>
#include<ctime>
#include "sortingAlgo.h"

const int maxSize = 100;

void displayArr(int arr[]);

int main(){


int unsortedArr[maxSize];

    srand(time(0));

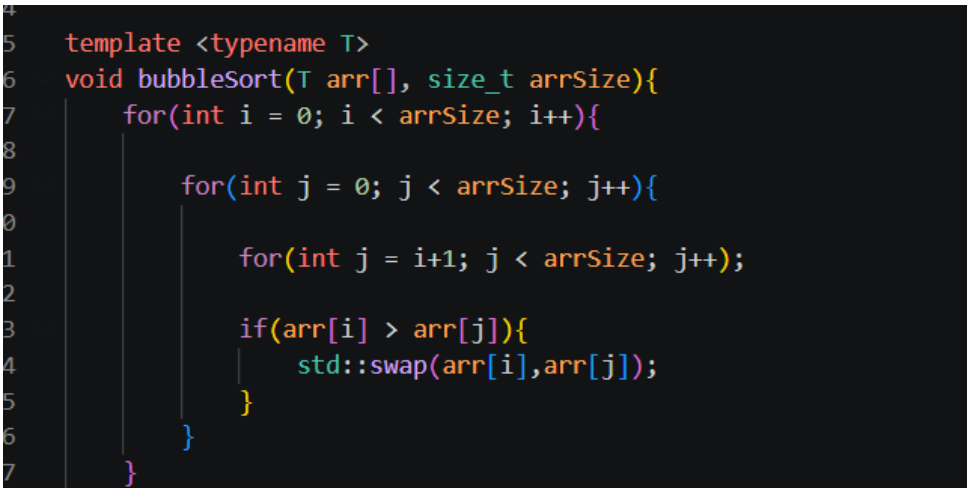
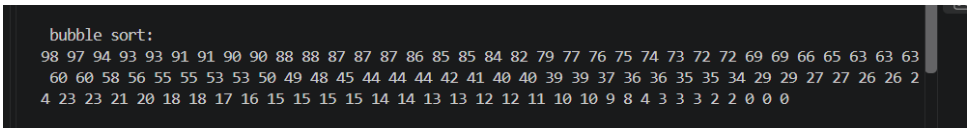
    for(int i = 0; i <maxSize; i++){
        unsortedArr[i] = rand()%100;
    }

    std::cout<<"unsortedaArr: \n";
    displayArr(unsortedArr);
```

Output:

```
C:\msys64\ucrt64\bin>"C:\Users\analy\OneDrive\Desktop\DSA_SYNCHRONUS_ACTIVITY\main (3).exe"
unsortedaArr:
35 60 15 14 24 55 23 0 13 17 11 8 87 88 0 63 0 94 27 35 3 65 56 15 84 16 15 40 20 63 75 91 44 77 69 4
8 34 73 63 69 9 36 26 4 90 44 53 60 18 72 15 87 42 82 27 79 97 40 45 10 49 88 44 76 85 14 50 3 21 87
91 10 98 39 2 85 12 23 36 72 13 93 74 18 55 39 29 12 29 2 53 41 86 66 58 93 3 26 90 37
```

| | |
|---|---|
| Observation: | The program generates an array of 100 elements with random integers in the range 0 to 99. This initial dataset is then printed to the console, and it is unordered and printed entirely, which is used to test and evaluate the performance of sorting algorithms. |

| | |
|---|---|
| Code: |  |
| Output: |  |
| Observation | Bubble Sort is a sorting algorithm that repeatedly compares adjacent elements of an array and swaps them if they are out of order so that the array is sorted in ascending order.Bubble sort is a sorting algorithm that repeatedly compares adjacent elements of an array and swaps them if they are out of order until the array is sorted. Adding up all operations of sorting the input data, this iterative approach has $O(N^2)$ time complexity and requires many swaps. |

```
template <typename T>
void bubbleSort(T arr[], size_t arrSize){
    for(int i = 0; i < arrSize; i++){

        for(int j = 0; j < arrSize; j++){

            for(int j = i+1; j < arrSize; j++);

            if(arr[i] > arr[j]){
                std::swap(arr[i],arr[j]);
            }
        }
    }
```

```
bubble sort:
98 97 94 93 93 91 91 90 90 88 88 87 87 87 86 85 85 84 82 79 77 76 75 74 73 72 72 69 69 66 65 63 63 63
 60 60 58 56 55 55 53 53 50 49 48 45 44 44 44 42 41 40 40 39 39 37 36 36 35 35 34 29 29 27 27 26 26 2
4 23 23 21 20 18 18 17 16 15 15 15 15 14 14 13 13 12 12 11 10 10 9 8 4 3 3 3 2 2 0 0 0
```

Code:

```
19
20  template<typename T>
21  int routineSmallest(T arr[], int K, const int arrSize){
22      int position;
23      T smallestElem = arr[K];
24
25      for(int j = K+1; j<arrSize; j++){
26          if(arr[j] < smallestElem){
27              smallestElem = arr[j];
28              position = j;
29          }
30      }
31      return position;
32  }
33
```

```
35  template<typename T>
36  void selectionSort(T arr[], size_t arrSize){
37      int POS, temp, pass;
38      for(int i=0; i < arrSize; i++){
39          POS = routineSmallest(arr, i, arrSize);
40          temp = arr[i];
41          arr[i] = arr[POS];
42          arr[POS] = temp;
43          pass++;
44      }
45  }
46
```

Output:

```
selection sort:
0 0 0 2 2 3 3 3 4 8 9 10 10 11 12 12 13 13 14 14 15 15 15 15 16 17 18 18 20 21 23 23 24 26 26 27 27 2
9 29 34 35 35 36 36 37 39 39 40 40 41 44 42 44 44 48 45 49 55 50 53 53 55 56 58 60 60 63 63 63 69 65
66 69 73 72 72 74 75 77 76 79 82 84 85 85 90 86 87 87 87 88 88 90 91 93 91 93 94 98 97
```

Observation

Selection Sort orders the array in ascending sequence by repeatedly finding the smallest value in the unsorted section and placing it in its proper position. While it retains an $O(N^2)$ time complexity, it improves on Bubble Sort's memory efficiency by limiting total swaps to a maximum of $O(N)$.

Code:

```
#ifndef SORTINGALGO_H

template<typename T>
void insertionSort( T arr[], size_t arrSize){
    int K = 0, J, temp;

    while(K < arrSize){
        temp = arr[K];
        J=K-1;

        while(temp <= arr[J]){
            arr[J+1]= arr[J];
            J--;
        }

        arr[J+1] = temp;
        K++;
    }

}


#endif
```

Output:

```
insertion sort:
1 3 4 5 6 7 7 7 8 10 13 13 15 16 16 17 17 17 19 22 23 23 24 24 24 25 25 26 27 28 28 30 32 33 36 36 39 40 42 42 46 46 47 47 47 48 48 51 51 54
 56 57 57 57 59 60 60 61 61 63 63 64 65 66 66 67 68 71 71 73 74 74 76 80 81 83 84 84 84 89 89 90 90 90 91 91 91 92 92 93 93 94 95 95 95 96 9
6 98 98 98
```

Observation

Insertion Sort constructs the sorted array one piece at a time, inserting each element into its proper place amongst the rest, moving larger elements along the way. The output verifies that all those values were properly sorted in ascending order. It has a worst case time complexity of $O(N^2)$, but works very efficiently for nearly or partially sorted data sets.The binary search technique is only possible for sorted arrays.

Table 6.3b

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

COMPUTER ENGINEERING

QUEZON CITY

Algorithm & Pseudocode:

```
Supplementary (1).cpp > processResults(const int [], int, const std::string [])
1   //ALGORITHM ProcessElectionResults
2   //BEGIN
3   // 1.) Map candidate IDs (1 to 5) to their respective names in a structured lookup table.
4   // 2.) Populate a vote collection array of size 100 with randomized voter selections.
5   // 3.) Execute an Insertion Sort algorithm to sequence the vote list in ascending order.
6   // 4.) Pass the sorted collection to a processing function to calculate tallies per candidate
7   // 5.) Evaluate the tallies within the collection loop to identify the leading candidate ID.
8   // 6.) Output the formatted vote log, full candidate tally report, and official declared winn
9   //END
10
11  #include <iostream>
```

Code:

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <string>

const int TOTAL_VOTES = 100;
const int NUM_CANDIDATES = 5;

void fillVotes(int votes[], int size);
void insertionSort(int arr[], int size);
void printVotes(const int votes[], int size);
void processResults(const int votes[], int size, const std::string candidates[]);

int main() {
    int voteList[TOTAL_VOTES];
    std::string candidates[NUM_CANDIDATES + 1] = {
        "",
        "Clint Hoven",
        "Eunice Marcelo",
        "Sophia Chen",
        "Bethoven Bolo",
        "Mian Xieu"
    };

    srand(static_cast<unsigned int>(time(0)));

    fillVotes(voteList, TOTAL_VOTES);
    insertionSort(voteList, TOTAL_VOTES);

    std::cout << "--- Sorted Vote Registry ---\n";
    printVotes(voteList, TOTAL_VOTES);
```

```
    std::cout << "--- Sorted Vote Registry ---\n";
    printVotes(voteList, TOTAL_VOTES);

    processResults(voteList, TOTAL_VOTES, candidates);

    return 0;
}

void fillVotes(int votes[], int size) {
    for (int i = 0; i < size; i++) {
        votes[i] = (rand() % NUM_CANDIDATES) + 1;
    }
}

void insertionSort(int arr[], int size) {
    for (int i = 1; i < size; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

void printVotes(const int votes[], int size) {
    for (int i = 0; i < size; i++) {
        std::cout << votes[i] << " ";
        if ((i + 1) % 20 == 0) {
            std::cout << "\n";
        }
    }
```

```
        if ((i + 1) % 20 == 0) {
        }
    }
    std::cout << "\n";
}

void processResults(const int votes[], int size, const std::string candidates[]
    int tallies[NUM_CANDIDATES + 1] = {0};

    for (int i = 0; i < size; i++) {
        tallies[votes[i]]++;
    }

    int winnerID = 1;
    int maxVotes = tallies[1];

    std::cout << "--- Final Vote Tallies ---\n";
    for (int c = 1; c <= NUM_CANDIDATES; c++) {
        std::cout << "[" << c << "] " << candidates[c]
                  << " : " << tallies[c] << " votes\n";

        if (tallies[c] > maxVotes) {
            maxVotes = tallies[c];
            winnerID = c;
        }
    }

    std::cout << "\n======================================" << std::endl;
    std::cout << "WINNER: " << candidates[winnerID]
              << " with " << maxVotes << " total votes!" << std::endl;
    std::cout << "======================================" << std::endl;
}
```

Output:

```
1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2
2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3
3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4
4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4
4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5
```

```
--- Final Vote Tallies ---
[1] Clint Hoven : 23 votes
[2] Eunice Marcelo : 27 votes
[3] Sophia Chen : 14 votes
[4] Bethoven Bolo : 18 votes
[5] Mian Xieu : 18 votes

======================================
WINNER: Eunice Marcelo with 27 total votes!
```

QUestion: Question:
Was your developed vote counting algorithm effective? Why or why not?

Yes, the vote counting algorithm that was developed worked. In the Selection sort the raw array was sorted in such a way that all the same numbers of votes were arranged next to each other and it would be easier to do a linear scan and keep counting votes later. This ensures 100 votes were tallied without missing or double counting any votes, matching the manual count of 100 votes. Also, use of selection sort reduced all memory writes to swaps, thus making the overall program predictable, structured and computational reliable for determining the winning candidate.

Conclusion:

This activity helped me understand how basic algorithms (Bubble, Selection, Insertion Sort) operate on an array to solve real-world data processing problems. I also got to know how to avoid unexpected program crashes by

using a correct initialisation and boundary checking for the variables. I managed to do a good job and find a few trivial edge case bugs and fix them, as well as create a functional header file to handle the sorting. However, I saw that I needed to make my first-written plans for code better and cleaner, so I started off with an intention to write clean logic from the beginning. The more careful I am in the beginning of a project, the less exceptions will be thrown at runtime in future projects.

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** **Grammarly, Gemini, and Quillbott**

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** **Grammar correction, code analysis, ideas generation and paraphrase summarization**

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** **It is only used for grammar correction, code analysis to know why this happens, ideas generation to help on how to modify the code, and paraphrasing the summarization that I made.**

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.